AWS API Gateway Lab Part 2: Create HTTP API and Perform Testing

To Begin with the Lab

Summary of the Lab

In this part of the lab, an **HTTP API** was created in **Amazon API Gateway** to expose the backend **AWS Lambda** functions as public API routes. A new API named UserAPI was set up, and routes were mapped so that POST /user triggered createUserFunction, GET /user/{userId} triggered getUserFunction, and DELETE /user/{userId} triggered deleteUserFunction. This made **API Gateway** the bridge between the frontend or Postman and the Lambda backend. The lab then tested the API end to end by sending create, read, and delete requests and confirming the expected changes in **Amazon DynamoDB**. It also confirmed the missing-record case by returning User not found after deletion.

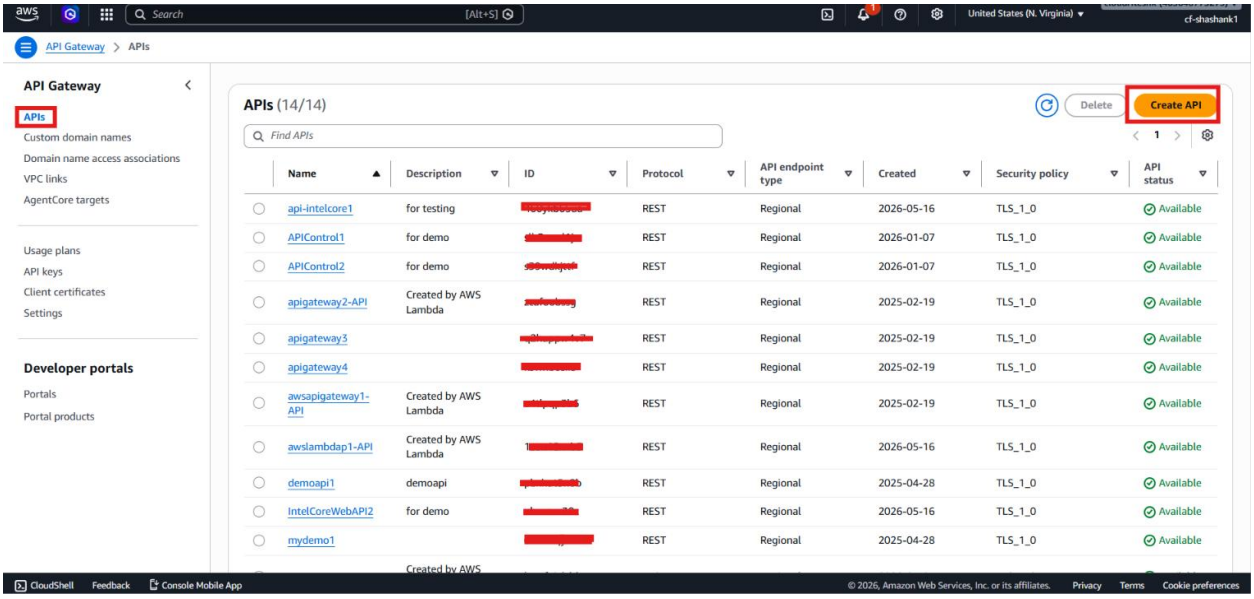
Understanding API Gateway in This Lab

API Gateway provides a secure HTTP entry point for clients so they do not call **AWS Lambda** directly. In this lab, it receives request methods like POST, GET, and DELETE, then forwards each one to the correct Lambda function. This keeps the application organized and makes the backend easier to scale and maintain. Using an **HTTP API** also gives a lighter and faster setup for Lambda-based routes compared with a full REST API.

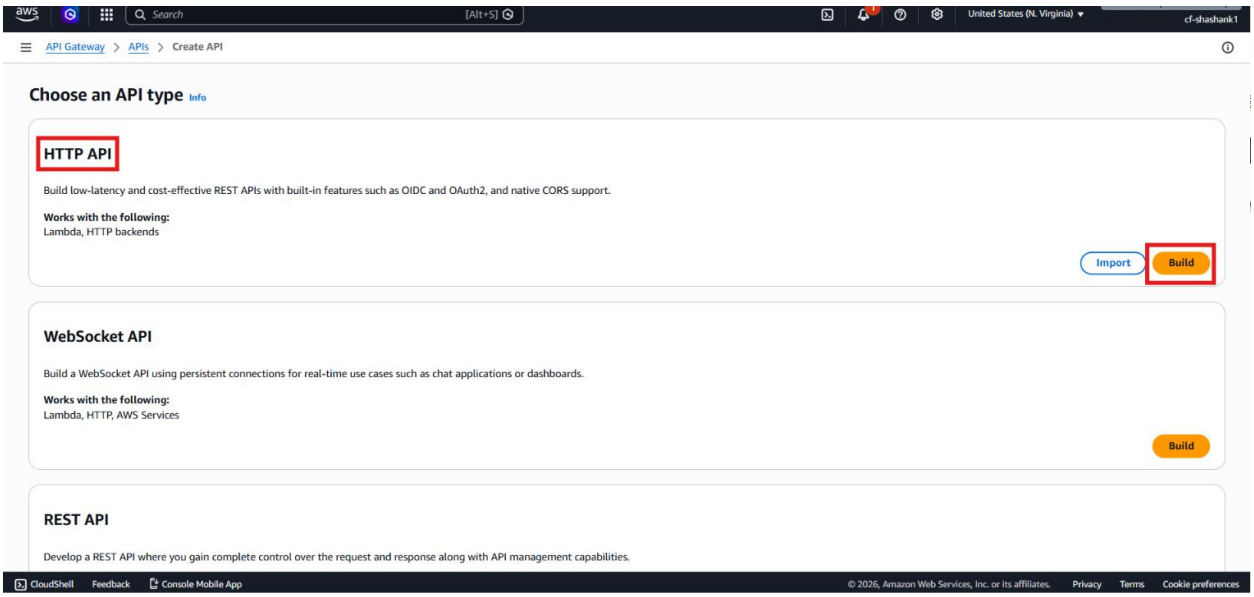
Lab Steps

Step 1 – Open API Gateway

- Sign in to the AWS Management Console.
- Open **API Gateway**.
- Click **Create API**.

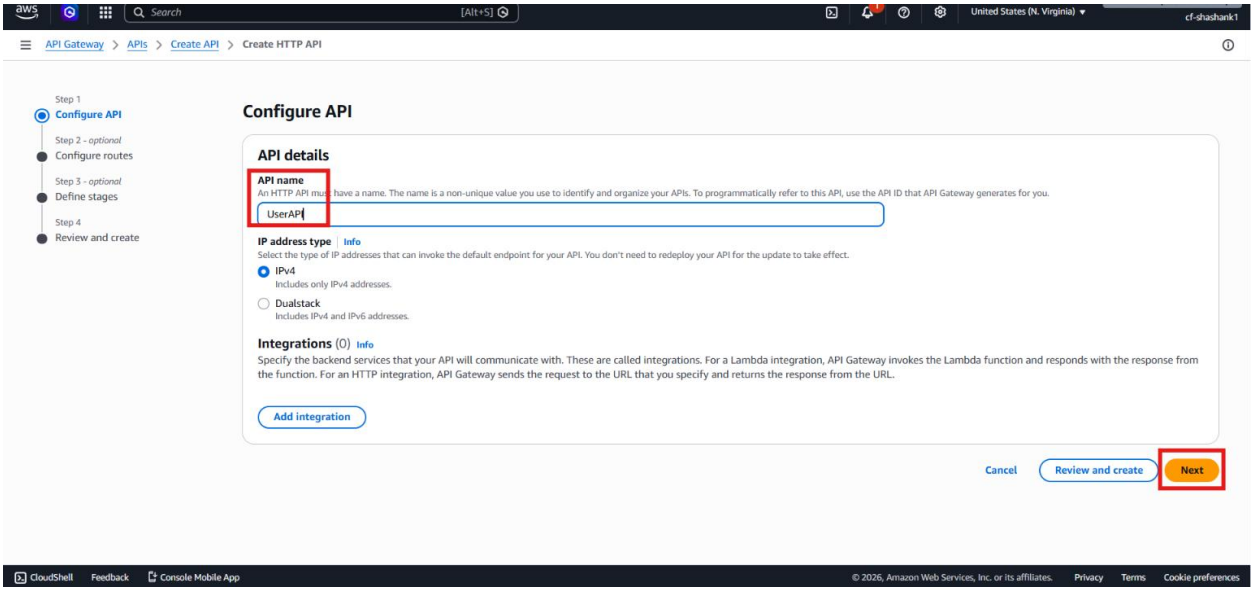


- Choose **HTTP API**.
- Click **Build**.

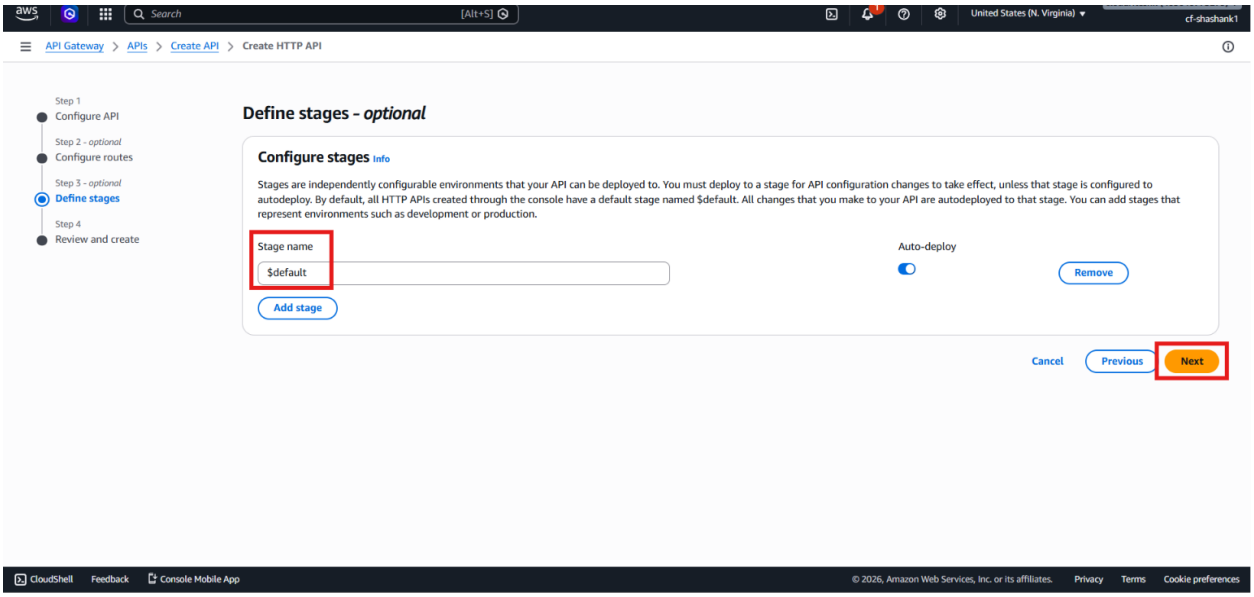


Step 2 – Create the HTTP API

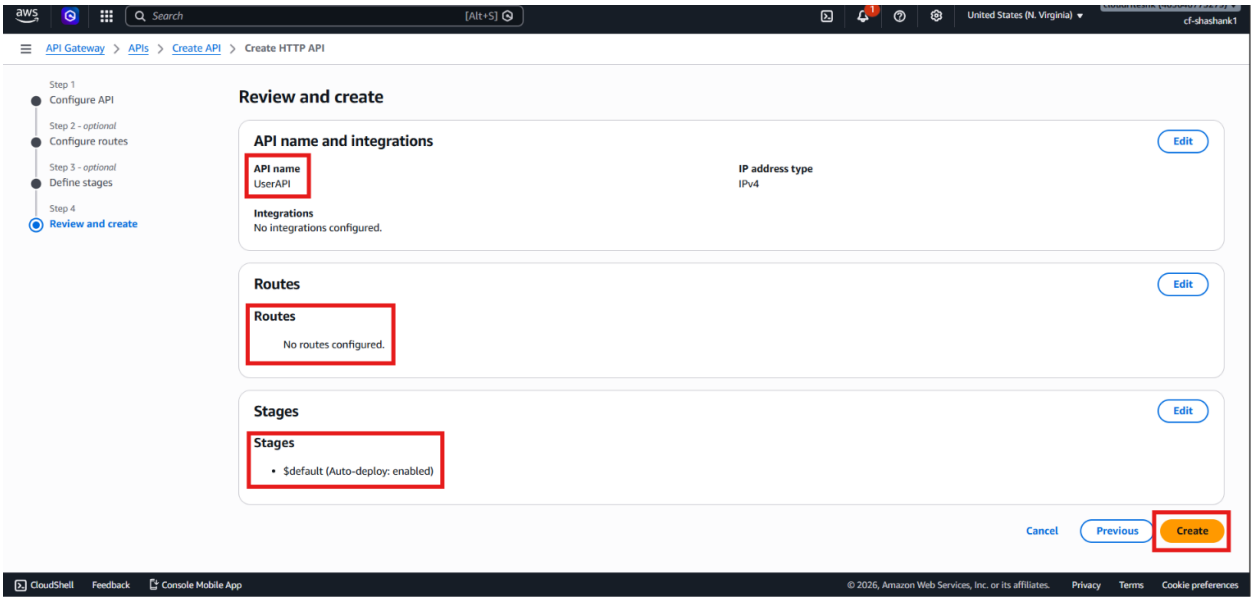
- Enter the API name: UserAPI
- Keep the default settings.



- Leave routes and stages unchanged for now.
- Click **Next**.

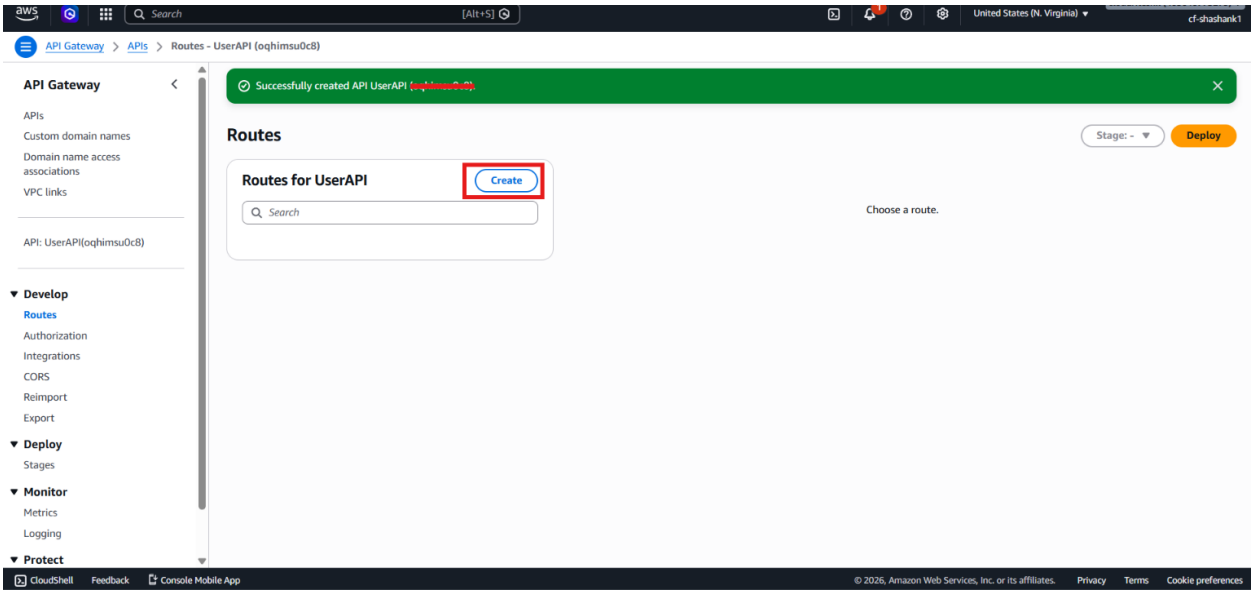


- Click **Create**.

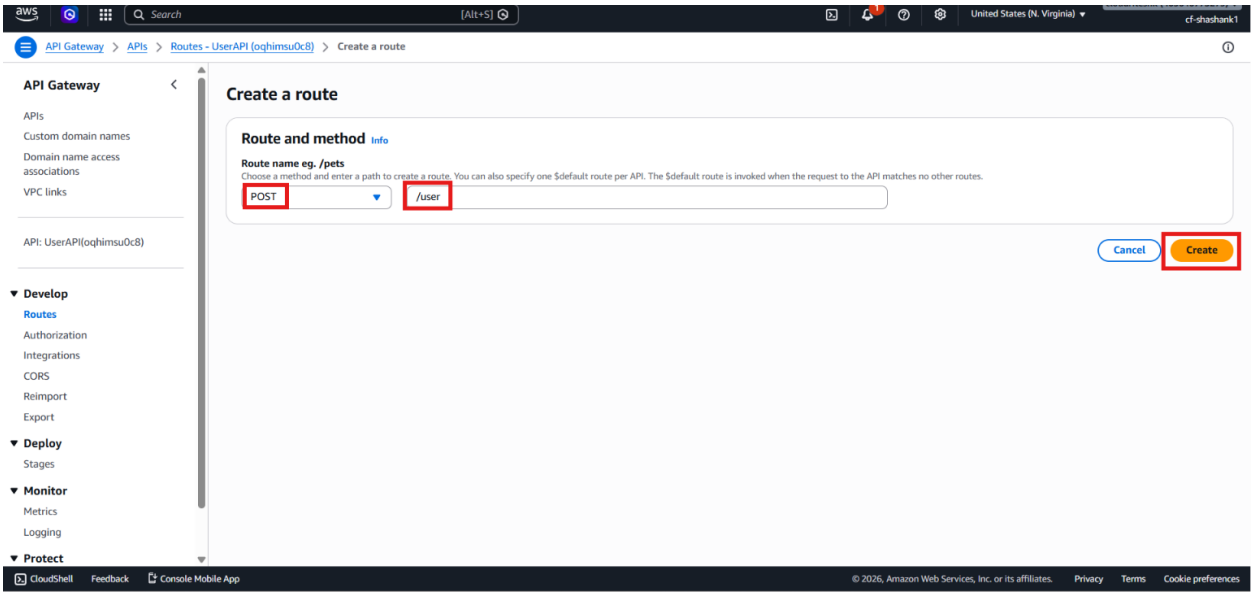


Step 3 – Create the Create User Route

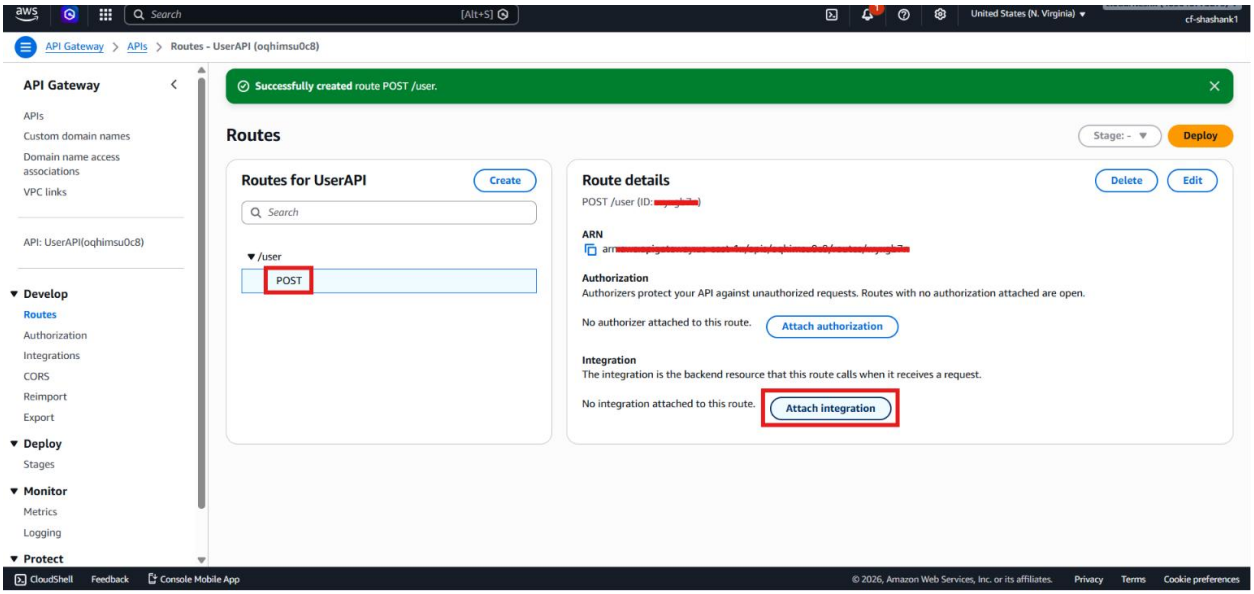
- Open **UserAPI**.
- Go to **Routes**.
- Click **Create**.



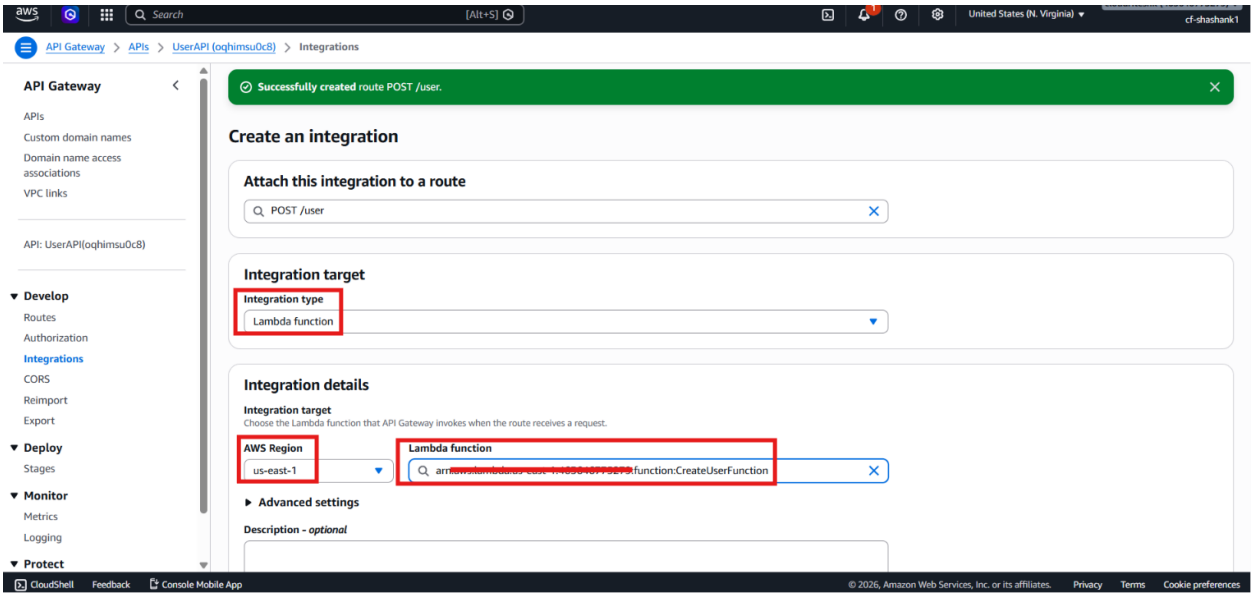
- Select **POST** as the method.
- Enter the route path: /user
- Click **Create**.



- Click **Attach integration**.



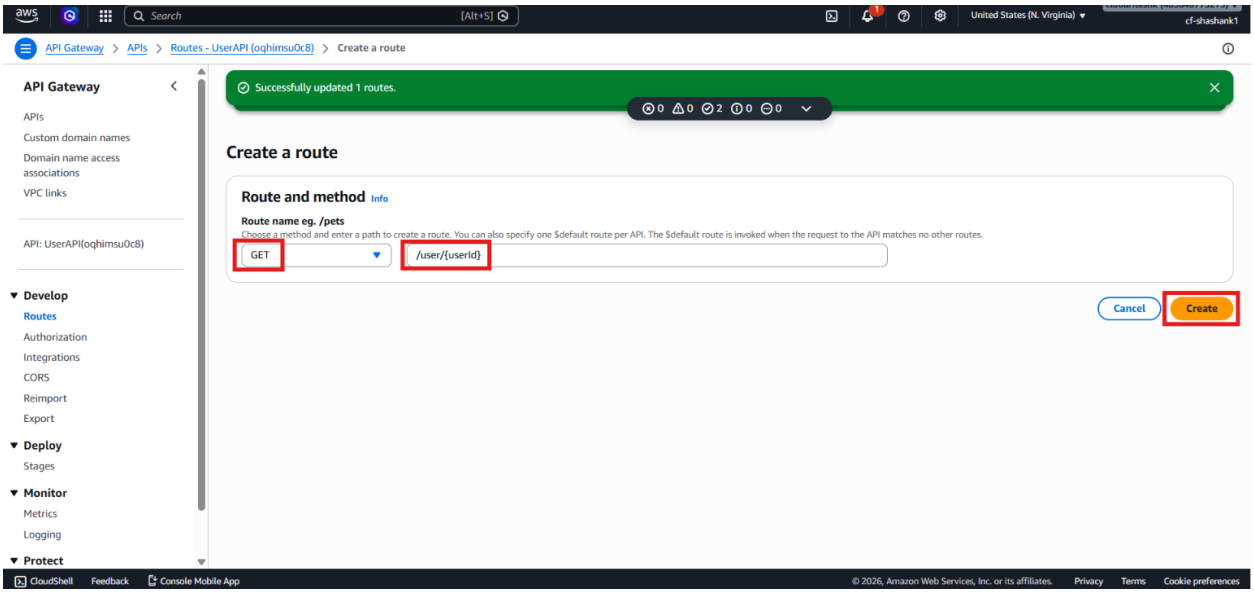
- Choose **Lambda function**.
- Select **createUserFunction**.



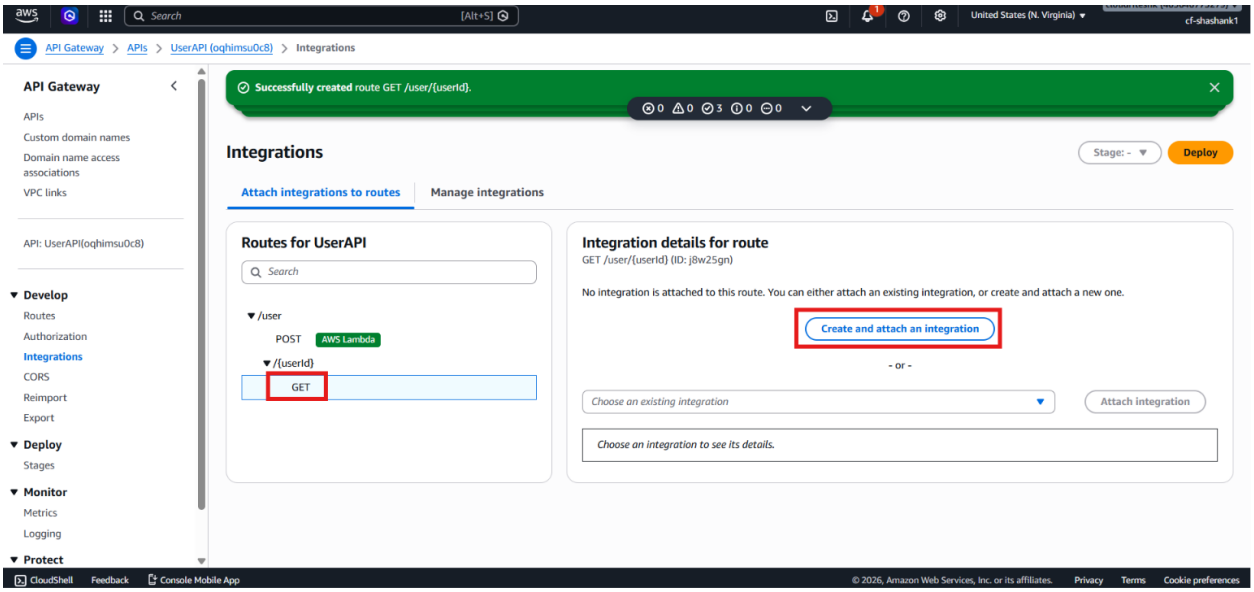
- Click **Create**.

Step 4 – Create the Get User Route

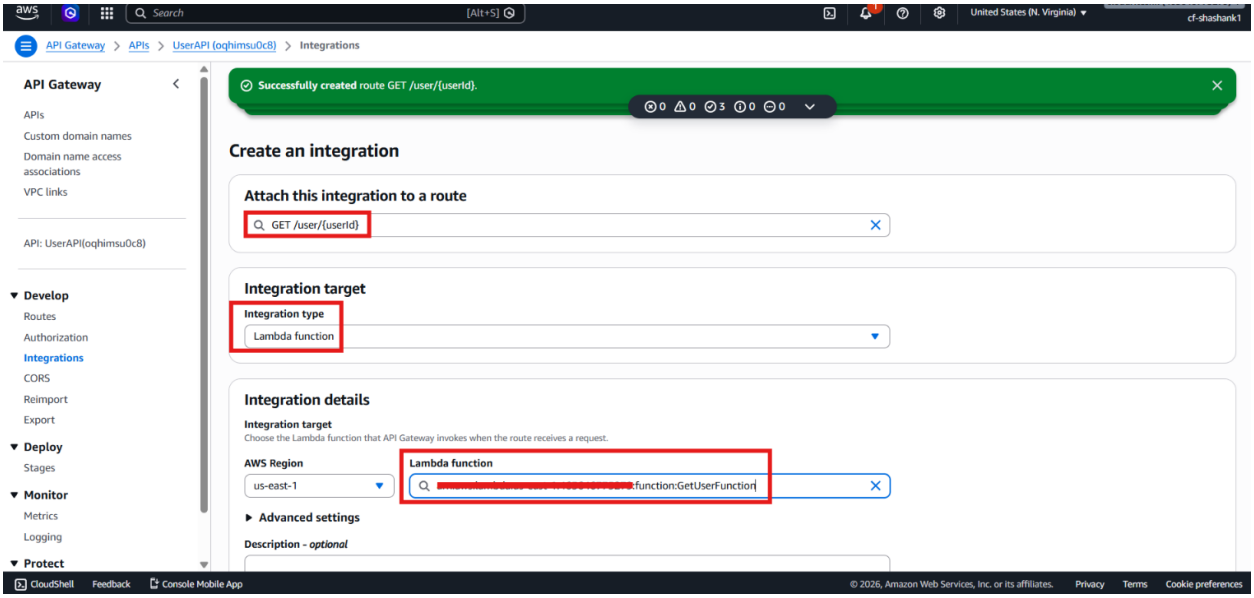
- Go back to **Routes**.
- Click **Create**.
- Select **GET** as the method.
- Enter the route path: /user/{userId}



- Click **Create**.
- Click **Attach integration**.



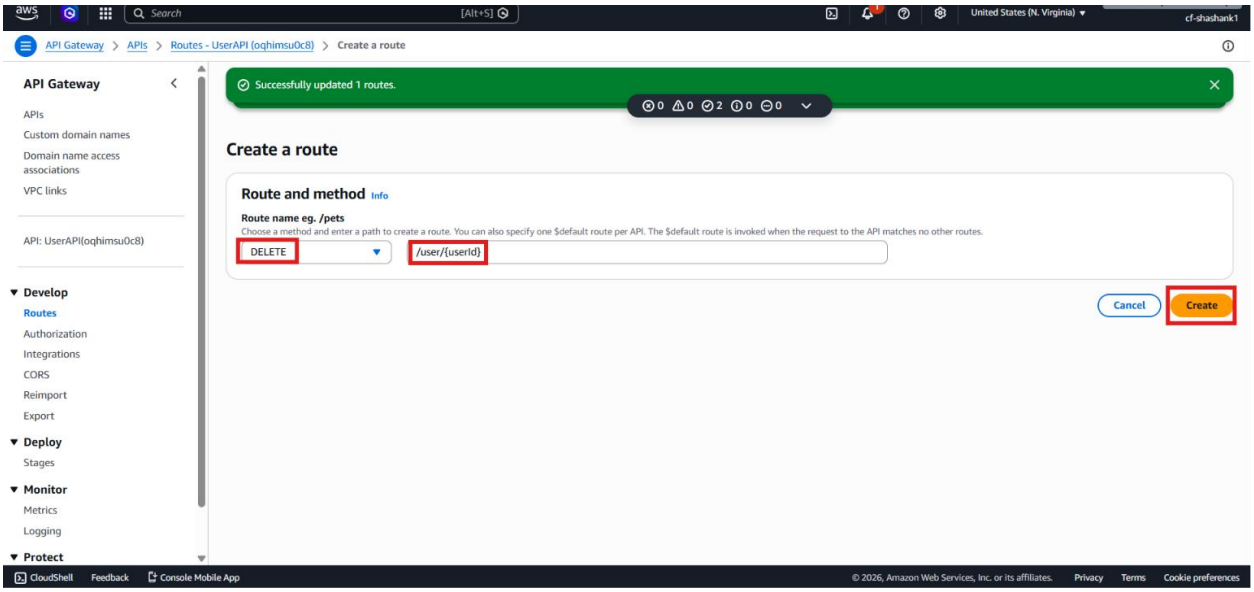
- Choose **Lambda function**.
- Select **getUserFunction**.



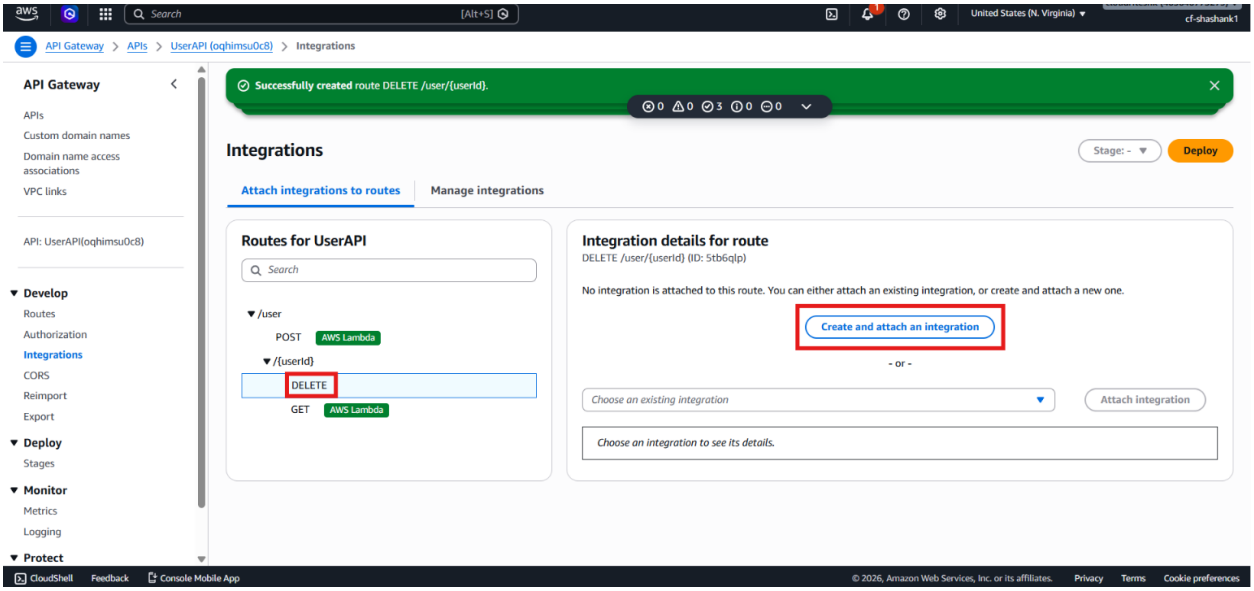
- Click **Create**.

Step 5 – Create the Delete User Route

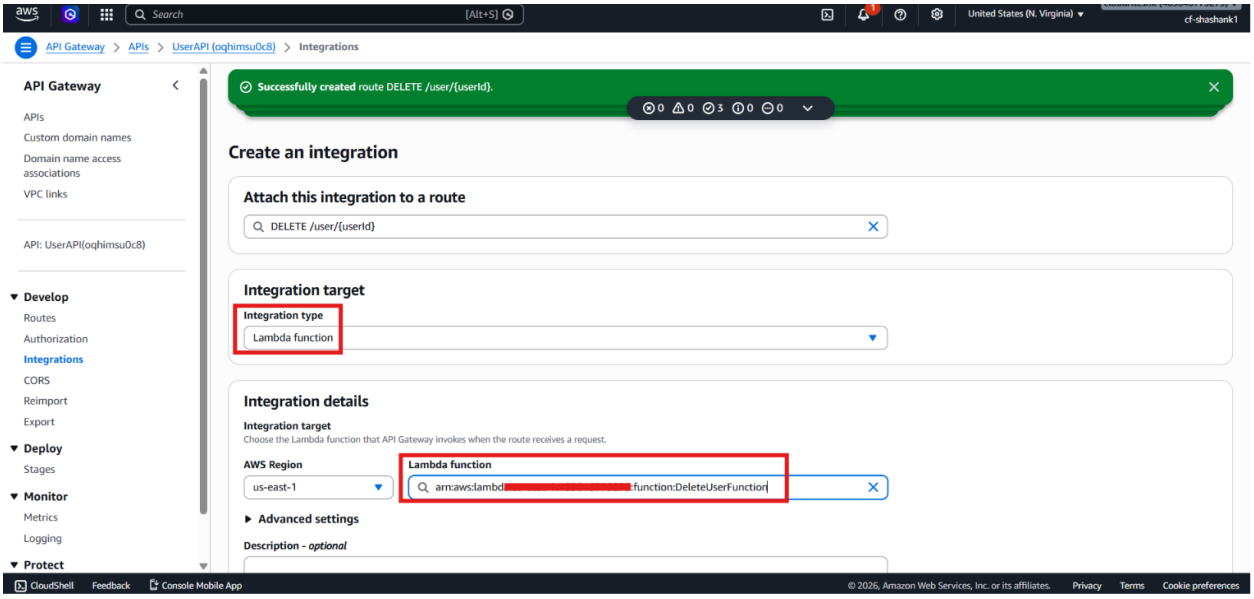
- Go back to **Routes**.
- Click **Create**.
- Select **DELETE** as the method.
- Enter the route path: /user/{userId}



- Click **Create**.
- Click **Attach integration**.



- Choose **Lambda function**.
- Select **deleteUserFunction**.



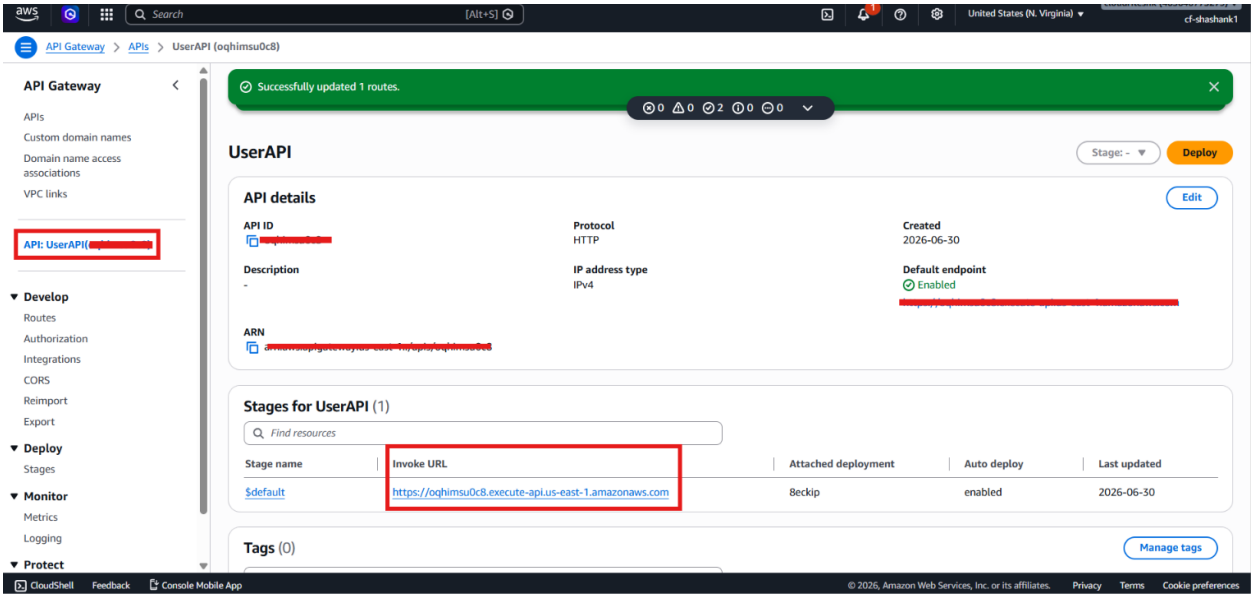
- Click **Create**.

Step 6 – Review the API Mapping

- Confirm the route mappings:
  - POST /user → **createUserFunction**
  - GET /user/{userId} → **getUserFunction**
  - DELETE /user/{userId} → **deleteUserFunction**
- Review the API trigger URL.
- Note that this is the URL the frontend or Postman will use to invoke the API.

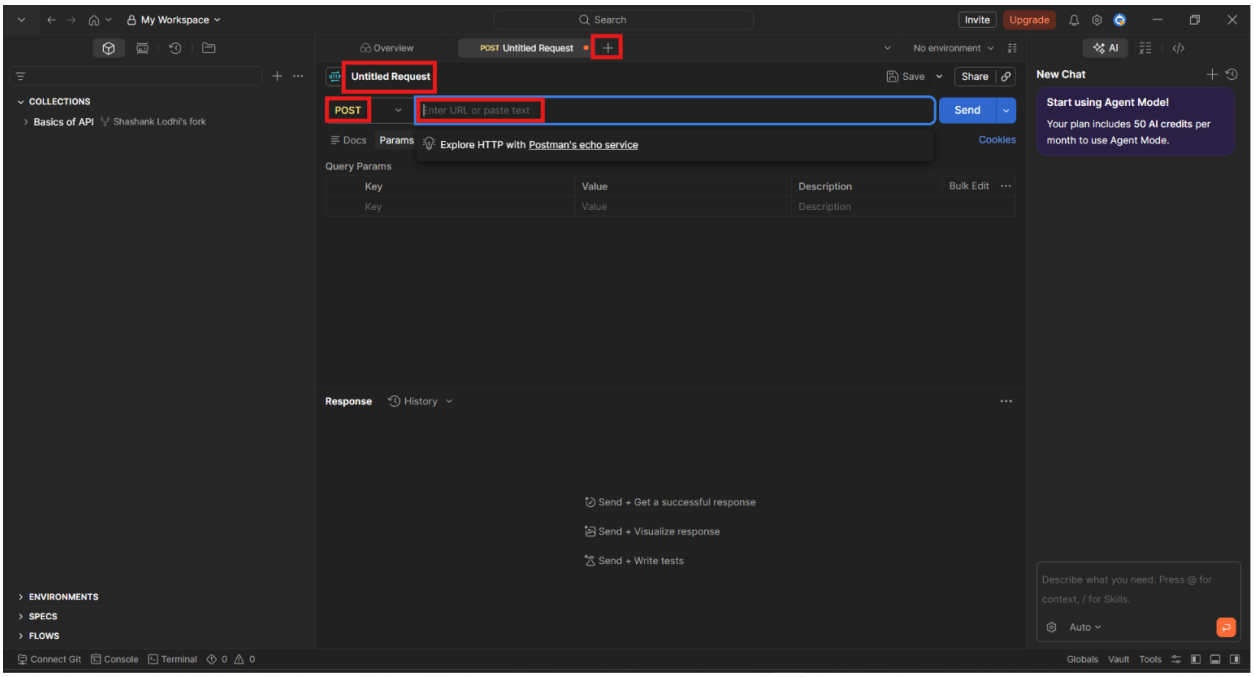
Step 7 – Open the API Gateway Invoke URL

- Sign in to the AWS Management Console.
- Open **API Gateway**.
- Select **UserAPI**.
- Go to **Stages**.
- Copy the **Invoke URL**.

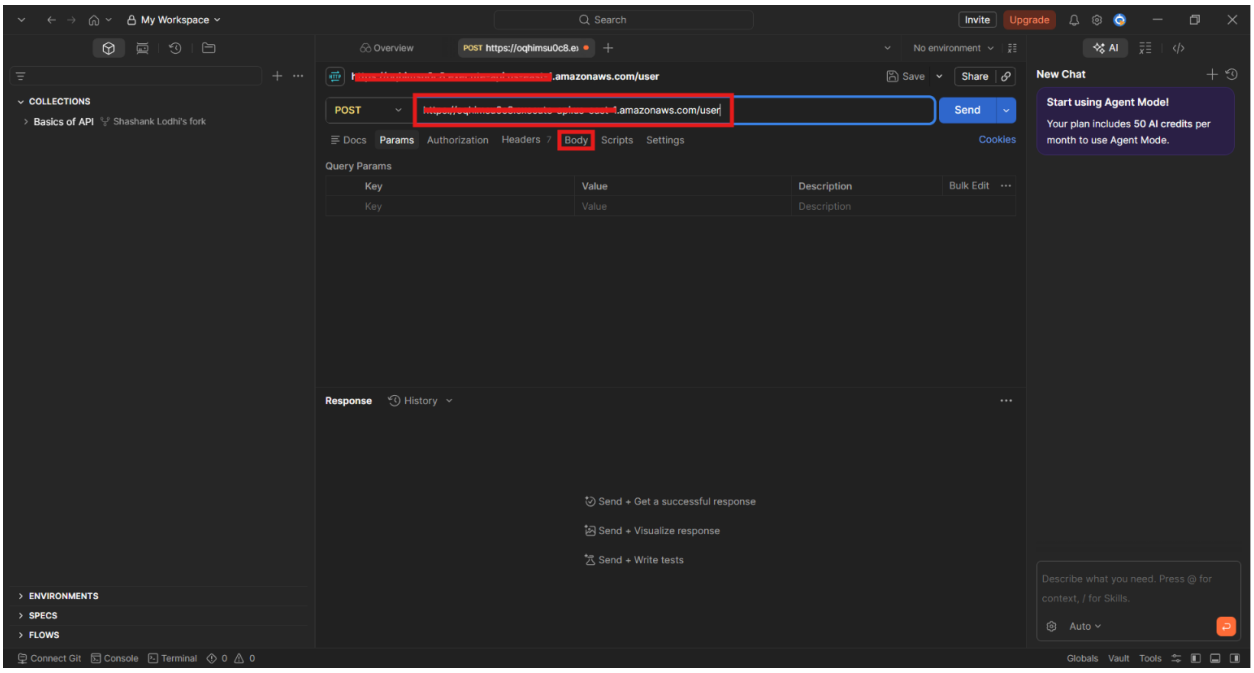


Step 8 – Test the Create User Request in Postman

- Open **Postman**.
- Create a new request.
- Set the method to **POST**.



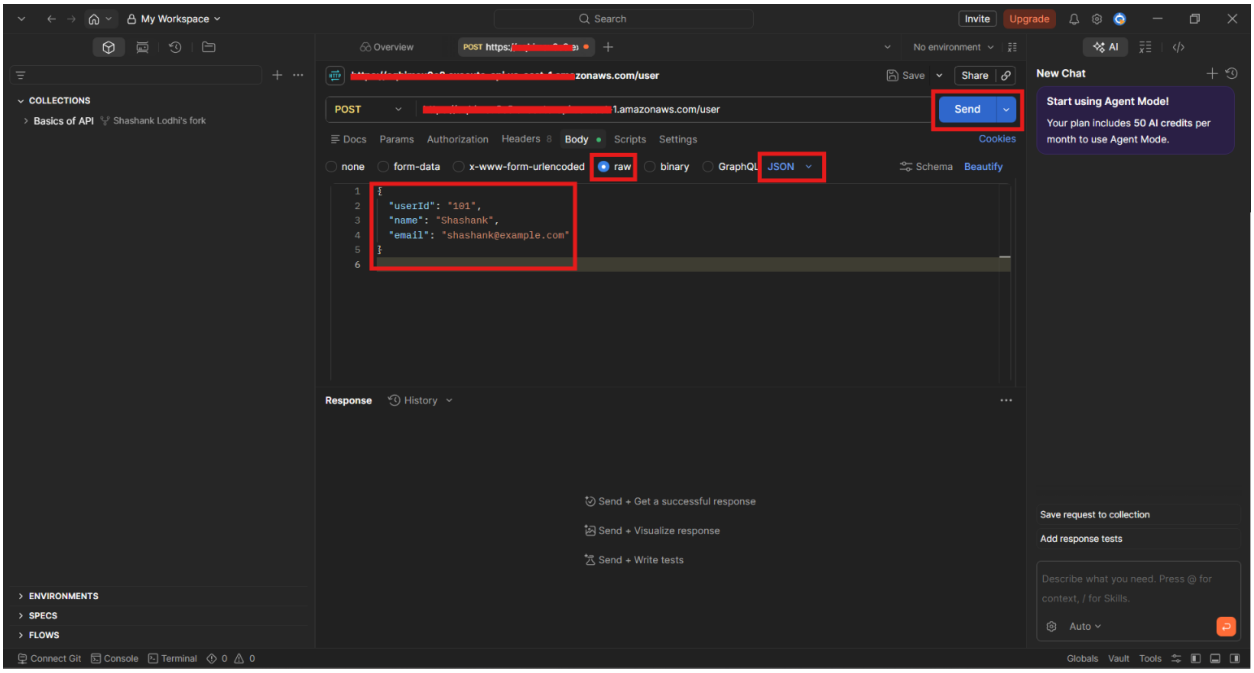
- Paste the invoke URL and append /user.
- Open the **Body** tab.



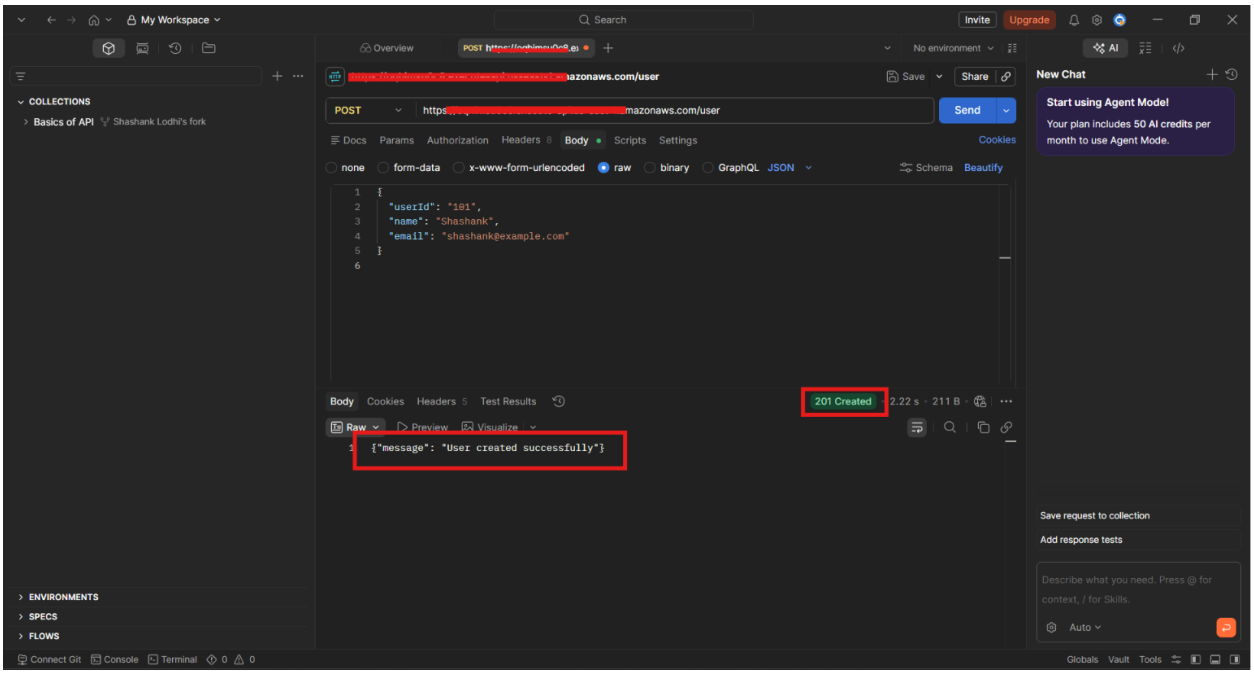
- Select **raw**.
- Choose **JSON** as the format.
- Add the user payload.

```
{
  "userId": "101",
  "name": "Shashank",
  "email": "shashank@example.com"
}
```

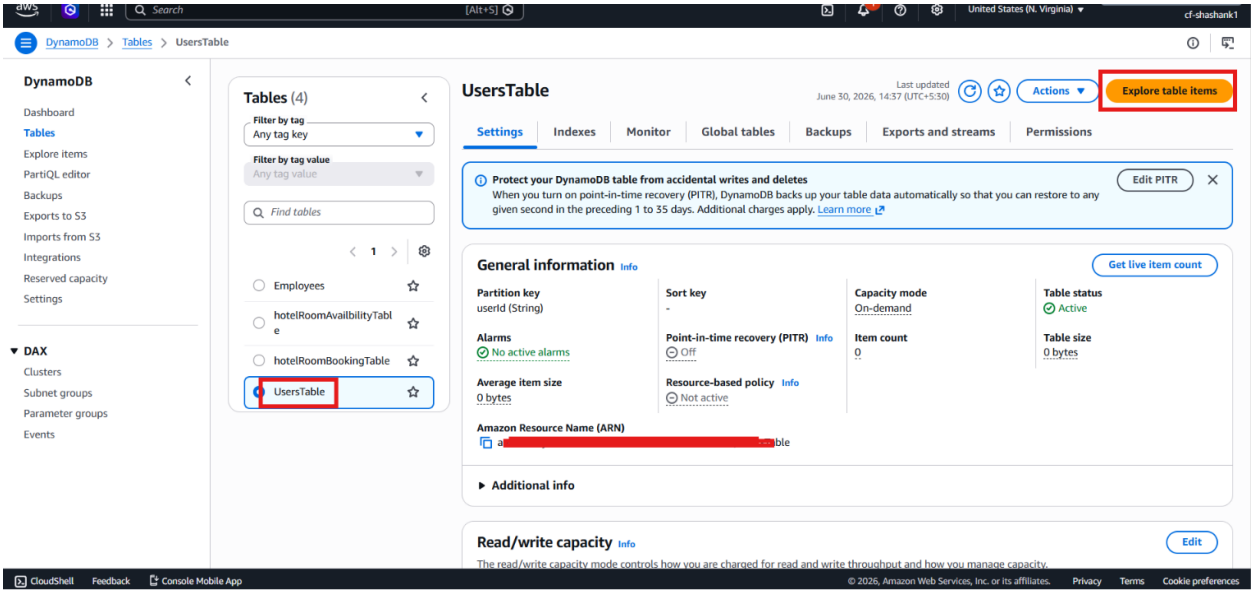
- Click **Send**.



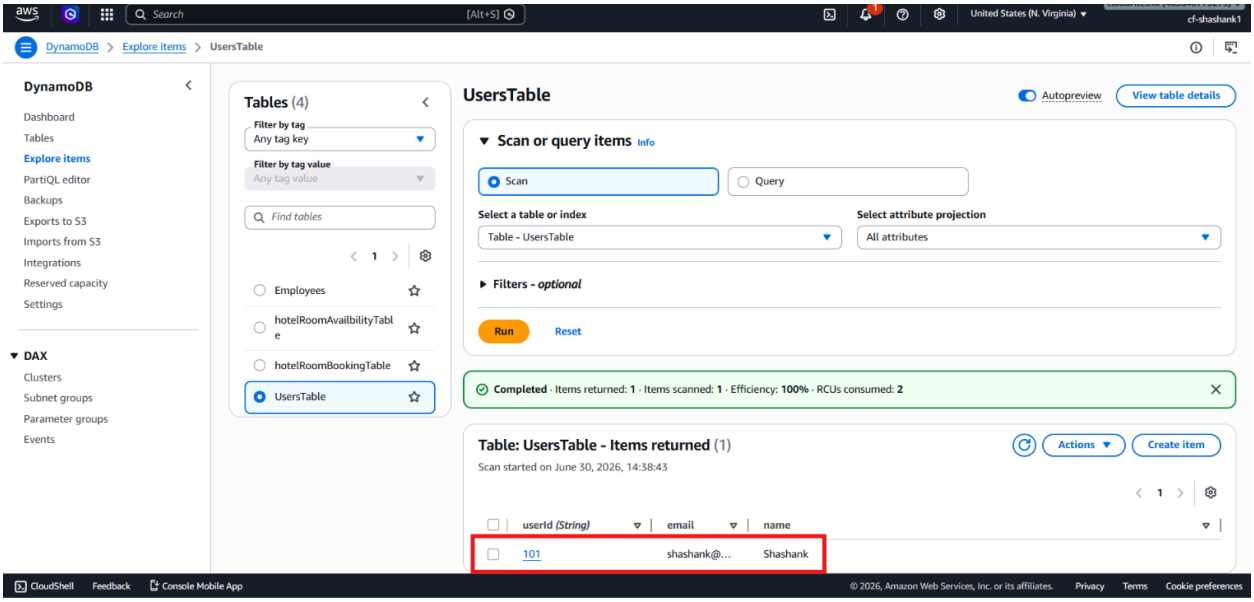
- Verify that the response shows user creation success.



- Open **Amazon DynamoDB**.
- Go to **Tables → UsersTable**.

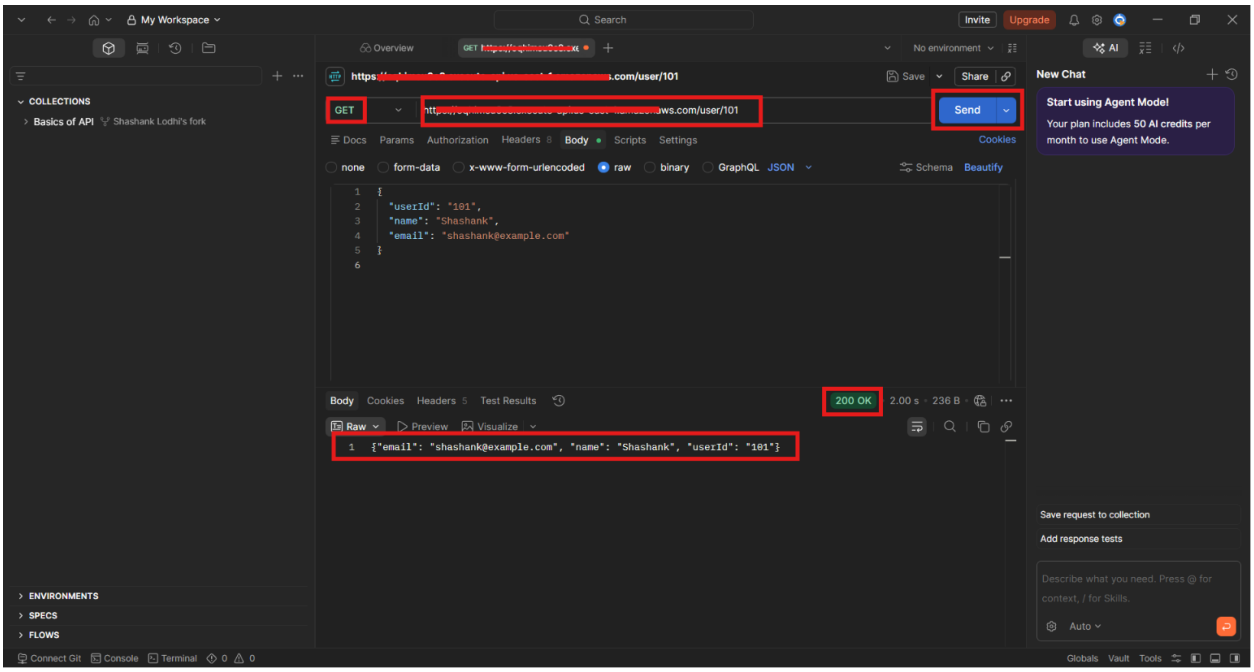


- Click **Explore table items**.
- Confirm that the record was inserted.



Step 9 – Test the Get User Request

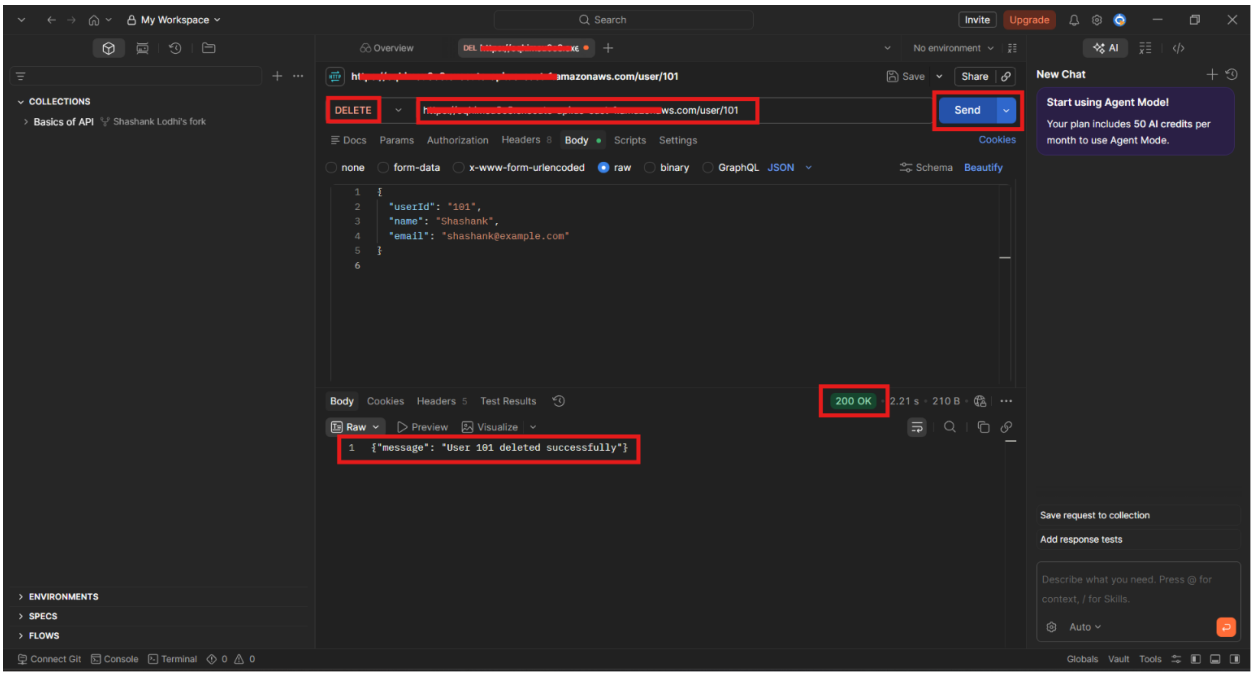
- In **Postman**, change the method to **GET**.
- Use the route /user/101.
- Click **Send**.
- Verify that the user details are returned.



- Confirm that the data matches the record stored in **UsersTable**.

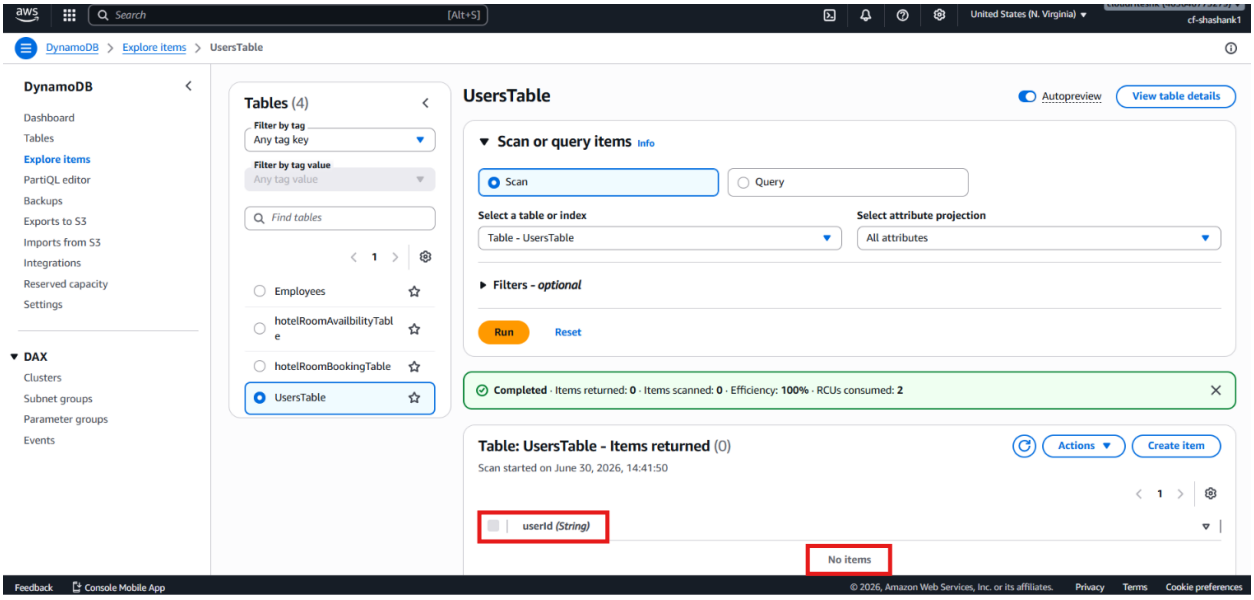
Step 10 – Test the Delete User Request

- In **Postman**, change the method to **DELETE**.
- Use the route /user/101.
- Click **Send**.
- Verify that the response confirms deletion.



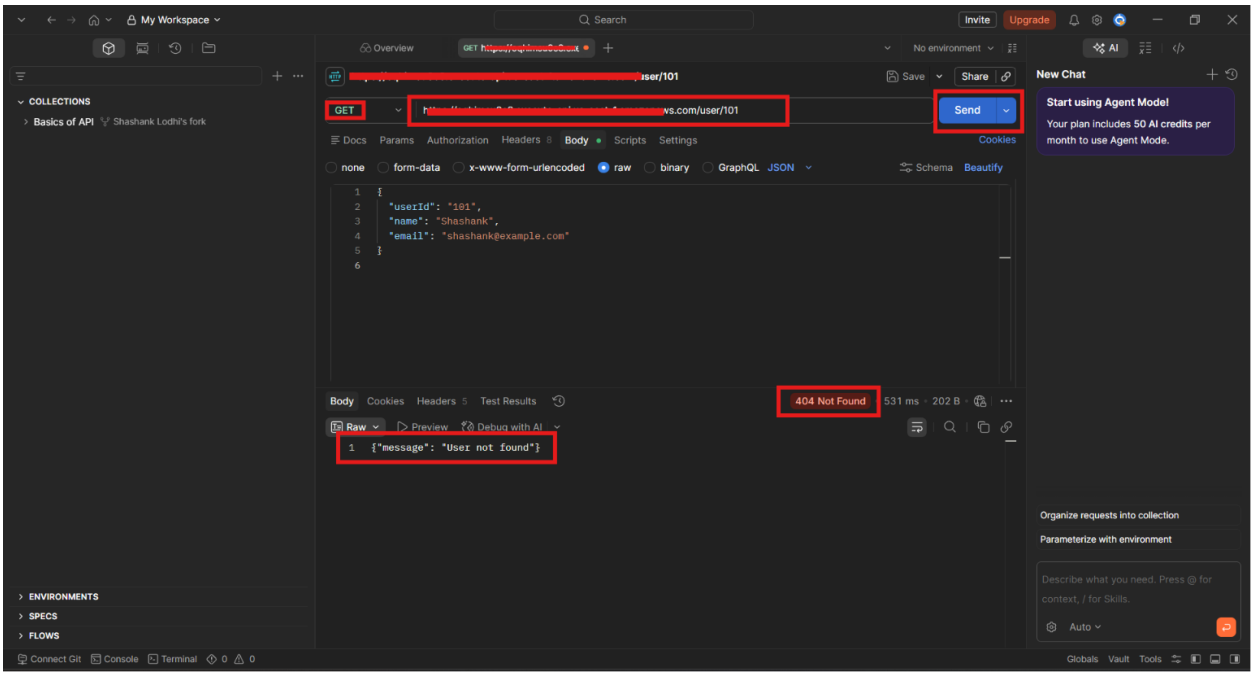
- Return to **Amazon DynamoDB**.
- Refresh **UsersTable**.

- Confirm that the record was removed.



Step 11 – Confirm the Error Case

- Send the same GET /user/101 request again after deletion.
- Verify that the response shows **User not found**.



- Confirm that the API correctly handles missing records.

Step 12 – Clean Up the API Gateway Resource

- Return to **API Gateway**.
- Select the **HTTP API** if you want to remove it.
- Delete the API Gateway resource only if the lab is complete.

- Keep the **Lambda** functions and **DynamoDB** table if you want to reuse them later.

Verification

- Verified that the **HTTP API** was created successfully in **Amazon API Gateway**.
- Verified that the route mappings were defined correctly for create, read, and delete operations.
- Verified that POST /user was connected to createUserFunction.
- Verified that GET /user/{userId} was connected to getUserFunction.
- Verified that DELETE /user/{userId} was connected to deleteUserFunction.
- Verified that the invoke URL was copied from the **Stages** section and used for testing.
- Verified that the POST request created a user record successfully in **Amazon DynamoDB**.
- Verified that the GET request returned the correct user details.
- Verified that the DELETE request removed the record from **Amazon DynamoDB**.
- Verified that a repeated GET /user/101 request after deletion returned User not found.
- Verified that the API was working as the bridge between the frontend and the Lambda backend.